Izrada rječnika za računalnu provjeru riječi hrvatskoga jezika Hunspellom

Krunoslav Šebetić | posljednja izmjena: listopad 2026. | kruno.se@gmx.com

Tablica sadržaja

Izrada rječnika za računalnu provjeru riječi hrvatskoga jezika Hunspellom 1
Što je pravopis, a što računalna provjera pravopisa 2
Hunspell nije isto što i pravila za provjeru pojedinog jezika 3
Anatomija pravila za provjeru ispravnosti napisanih riječi 4
Rječnička datoteka – relativno jednostavan popis riječi 4
Datoteka afiksā – logika iza rječnika 6
Prefiksi (PFX), sufiksi (SFX) i uvjeti u klasama 7
Crossproduct – istovremeno kombiniranje sufiksa i prefiksa 9
NEEDAFFIX – prijedlog za komparativ i superlativ 9
Kompresija klasa pomoću direktive AF 10
Tablica sustavnih pogrešaka: direktiva REP 11
Higijena ulaznih znakova: tablica ICONV 12
BREAK i WORDCHARS – rastavljanje riječi na određenom (pravopisnom) znaku 13
FLAG long – smisleni nazivi klasa 14
FULLSTRIP – zamjena cijelih riječi iz rječnika 15
SET, TRY, KEY – samo ukratko 15
Prijedlog rješenja za brojeve uz imenicu ili pridjev 16
Rad s Hunspellom iz naredbenoga retka na GNU/Linuxu 22
Dodatne napomene uz računalnu provjeru riječi hrvatskoga jezika 23
Što uvrstiti u rječnik, a što ne – i zašto 23
Tvorba naspram fleksije i granularnost 25
Označavanje rječnika za provjeru gramatike 27
Zašto je potrebno imati označen rječnik 28
Označavanje rječnika – tehnička strana priče 29
Označavanje rječnika – nama neistražen teritorij 30
Samo ukratko za kraj 31

Mogao sam ovaj dokument započeti na bezbroj načina, ali možda ga je najbolje otvoriti priznanjem kako sam ovako nešto, kada je bila prilika, trebao napisati prije desetak godina[1], ali tada nisam imao jasnu sliku o tome kako bi takav tekst trebao izgledati. Znao sam što i kako treba raditi praktično, ali nisam znao kome dokument namijeniti i kako ga strukturirati. Možda ne znam ni sada, ali ipak sam se odvažio jer će u protivnome ono što sam mukom skupljao po zakutcima svijeta otvorenoga koda ostati zakopano onoga dana kada zbog obaveza i drugih životnih prilika više neću biti u mogućnosti samostalno održavati rječnik i popratne alate.

To se događa već sada jer u svijetu umjetne inteligencije rječnici trebaju biti označeni, odnosno riječima trebaju biti dodijeljene morfološke oznake poput vrste riječi, padeža, roda, broja, glagolskog lica i drugo (engl. *Part-of-Speech tagging*), a takav posao nadilazi snagu pojedinca. Da bi se ovakav rječnik iskoristio za izradu jezičnoga modela koji razumije strukturu rečenice a onda i da pomogne korisniku bolje pisati, potrebno je imati kvalitetnu osnovu. Taj se posao ne može napraviti automatski jer ne postoji otvorena a označena baza riječi hrvatskoga jezika kojom bi se ovaj rječnik za računalnu provjeru riječi[2] mogao označiti računalnim kodom.

Prije umjetne inteligencije i jezičnih modela bilo je dovoljno unositi riječi u rječnik i provjeravati zatipke jer drugo nije ni postojalo. Sada kada je tehnologija ovdje, problem je u tome što hrvatski nema označen rječnik koji bi bio otvoren i dostupan svima. Dakle javila se potreba da se u sadašnji rječnik uvedu nove značajke. Međutim to ne mogu sam, a utrošak vremena da netko krene proučavati izradu rječnika za računalni provjernik riječi hrvatskoga jezika Hunspellom je tolik da je lakše naučiti pravilno pisati negoli se boriti s tehničkom stranom rječnika. Ovaj tekst nastaje kako bi se premostio jaz između želje i potrebe za boljim rječnikom i činjenice da ovo nije nimalo lagan zadatak, a literature ovoga tipa, čini mi se, jednostavno nema. Tekst pokušava, sa svim svojim nedostatcima, popuniti tu prazninu i olakšati ulazak novim održavateljima u svijet računalne provjere riječi Hunspellom u digitalnoj eri u kojoj dominira umjetna inteligencija.

Što je pravopis, a što računalna provjera riječi

U hrvatskom se za englesku riječ *spell checker* nepraktično koristi sintagma *računalna provjera pravopisa*. Međutim taj je izraz pogrešan iz nekoliko razloga, pa je važno reći da pravopis bilježi pravila kako nekim jezikom pisati te se u pravopisnom priručniku nalaze poglavlja o pisanju glasova *č, ć, đ* i *dž*, ali i skupine *ije* i *je*, pravilo o pisanju velikoga i maloga slova, zatim ono o sastavljenom i nesastavljenom pisanju; tu je i pravilo o upotrebi interpunkcijskih znakova, ali i pravilo o pisanju riječi iz stranih jezika te pravila za pisanje pokrata i kratica. Međutim *spell checker* nije računalno pomagalo koje preslikava pravopisni priručnik jer ne može obuhvatiti sva pravopisna pravila. Takav dodatak provjerava samo je li koja riječ – izolirana od konteksta – napisana ispravno. Zarez je nešto što računalna provjera pravopisa ne može ni dotaknuti, veliko i malo slovo može riješiti do neke mjere (*zagreb* naspram *Zagreb*, ali postoji problem kada se općoj *imenici višnja* suprotstavi vlastito ime *Višnja*), sastavljeno i nesastavljeno pisanje ne dolazi u obzir jer

[1] https://repository.ffri.uniri.hr/object/ffri:1583
[2] Zašto se koristi izraz *računalni provjernik riječi* vidi u *Što je pravopis, a što računalna provjera pravopisa*.

riječi provjerava izolirano od onoga što dolazi prije ili poslije, interpunkcija je potpuno zanemarena jer se riječi pravopisnom rječniku poslužuju lišene svih interpunkcijskih i pravopisnih znakova, dakle samo čiste riječi. Računalna provjera pravopisa nije ni *slovopis* jer taj pojam označava skup slova kojima se bilježi neki jezik. Računalni provjernik riječi ne definira piše li se koji jezik latinicom, ćirilicom ili kako drugačije, već samo provjerava postoji li ono što je korisnik unio tipkovnicom na popisu riječi u rječniku. Engleski se piše drugačije od hrvatskoga, no iako hrvatski ne prati doslovno fonološko pisanje, u engleskom je to izraženije. Tako *spell checker* pomaže onome tko piše na engleskom jeziku provjeriti je li koju riječ napisao ispravno, pa je značenje engleske riječi *spelling* nešto uže i vrlo specifično u odnosu na ono što bi značila hrvatska sintagma *provjera pravopisa*. Provjeriti pravopis može značiti provjeriti piše li netko slijedeći pravopisna pravila, dakle značenje sintagme bi zapravo bilo nešto šire. Izraz *pravopisni provjernik* me, koji nudi Institut za hrvatski jezik[3], ne zadovoljava jer i opet u sintagmi imamo *pravopis*, a rečeno je da ta riječ znači više negoli je u ovom slučaju potrebno. Predlažem izraz *računalni provjernik riječi* jer u sastavu imamo i *računalo* i *provjeru*, ali imamo i *riječ*, upravo ono što *spell checker* (donekle) može provjeriti.

Hunspell nije isto što i pravila za provjeru pojedinog jezika

Računalni provjernik Hunspell ne smije se poistovjećivati s računalnom provjerom riječi. Sâm Hunspell je računalni program, kôd koji može primiti i obraditi pravila za provjeru pojedinoga jezika. Važno je to razdvojiti jer dok program ima središnje mjesto razvoja[4], rječnici su raštrkani diljem Interneta.[5] Druge aplikacije integriraju Hunspell kao modul za provjeru ispravno napisanih riječi (GNU/Linux, Firefox, Chrome, Adobe, MacOS...), a na održavateljima jezičnih paketa pojedine aplikacije jest da zatraže uključivanje novijih inačica u svaku od tih aplikacija pojedinačno. Zato se trenutačno događa da Firefox ima koliko-toliko ažuriran rječnik dok Chrome ima inačicu rječnika koja je u trenutku pisanja ovoga teksta starija od desetljeća. Ovakvo stanje nije problem samoga Hunspella, već održavatelja jezičnoga paketa pojedine aplikacije. Iako se rječnik na https://github.com/krunose/hunspell-hr ažurira (relativno) redovito, netko ta pravila treba zapakirati za korištenje za svaku aplikaciju zasebno. Pripremanje pravila za integraciju u pojedinu aplikaciju rade različiti održavatelji. Osoba koja održava sâm rječnik nije ista osoba koja priprema ta pravila za integraciju u internetski preglednik Firefox, kao što ni ova potonja nije ona koja pakira ta pravila za alat OOONA[6] ili MacOS. Često se događa da netko zatraži uključivanje novije inačice rječnika u aplikaciju, ali onda zapusti taj jezični paket i on jednostavno vremenom zastari. Problem je u tome što svaki operacijski sustav ima posebno mjesto za spremanje datoteka s pravilima, ali onda i svaka aplikacija na računalu ima svoje mjesto za te datoteke. Može se dogoditi da na jednom računalu jedna inačica rječnika provjerava ispravnost napisanih riječi na razini operacijskoga sustava, druga inačica u kontekstu internetskog preglednika, a treća u obrađivaču teksta poput LibreOfficea. Znači, Hunspell može provjeravati ispravnost napisanih riječi

[3] Vidi izvor na https://bolje.hr/rijec/spelling-checker-gt-pravopisni-provjernik/116/
[4] https://github.com/hunspell/hunspell
[5] https://github.com/krunose/hunspell-hr
[6] OOONA je profesionalni mrežni alat za prevođenje i stvaranje podnatpisa u medijskim sadržajima.

samo ako se upari s pravilima za pojedini jezik, a ona se pak razvijaju neovisno o samom provjerniku.

Anatomija pravila za provjeru ispravnosti napisanih riječi

Do sada se o pravilima za provjeru kojega jezika pisalo općenito, vrijeme je da se pokaže kako ta pravila uopće izgledaju. Kako za svaki jezik tako i za hrvatski, pravila se sastoje od dvije obične tekstne datoteke. U jednoj se nalazi popis riječi s brojem na vrhu, koji predstavlja ukupan broj riječi u rječniku, a druga sadržava popis nastavaka (prefiksā i sufiksā) koje se dodaju riječima u rječniku kako bi se od riječi u rječniku napravili svi oblici te riječi, dakle od *rječnik* dobivamo *rječnika, rječniku, rječničke, rječnikom*. Čini se komplicirano, ali osnove je lako pohvatati.

Rječnička datoteka – relativno jednostavan popis riječi

U svome nazivu ova datoteka nosi oznaku jezika i države u kojoj se koristi jer španjolski u Španjolskoj i Meksiku nije posve isti. U kontekstu hrvatskoga jezika situacija je jednostavna: `hr_HR.dic`. Prva oznaka označava jezik (`hr`), druga označava državu (HR) u kojoj se taj jezik koristi. U prvom retku datoteke nalazi ukupan broj riječi u rječniku. Zatim slijedi abecedni popis riječi (lema) koje u kontekstu hrvatskoga dolaze u nominativu jednine za imenice i infinitivu za glagole. Pridjevi se pišu u nominativu jednine muškoga roda u neodređenom obliku (ako je moguće) i tako redom. Riječi se u ovoj datoteci unose onako kako ih nalazimo u svakom jednojezičnom rječniku hrvatskoga jezika.

```
3
rječnik/AA
trčati/BA
žena/AB
```

Ono što u rječničkoj datoteci treba objašnjavati jesu zastavice, odnosno klase. Iako Hunspell izvorno koristi englesku riječ *flags*, koristim riječ *klasa* pa onda i *deklinacijska klasa* u smislu *deklinacijski obrazac* ili *deklinacijska vrsta*, pa kada se uvede pojam *vrsta*, onda se nekako *klasa* nameće kao sinonim[7] jer Hrvatski jezični portal za *klasa* navodi: *skupina [...] zajedničkih svojstava*, a deklinacijski obrasci zbilja i čine takvu skupinu.

U gornjem primjeru imamo tri klase odvojene kosom crtom od riječi: `AA`, `AB` i `BA`. Te klase čine vezu između rječničke datoteke i datoteke s afiksima. Već je bilo spomena da se pravila sastoje od dvije datoteke, a ako je rječnička datoteka `hr_HR.dic`, onda je ova druga `hr_HR.aff`. Datoteka afiksā sadrži popis svih deklinacijskih i konjugacijskih nastavaka i obrazaca. Ako se u rječničkoj datoteci nalazi klasa `AA`, onda ista takva klasa mora imati svoju sekciju u datoteci afiksā a gdje su dane instrukcije Hunspellu kako deklinirati, recimo, *rječnik* po svim padežima u jednini i množini, ali o tome više u dijelu `Datoteka afiksā – logika iza rječnika`.

[7] https://hjp.znanje.hr/index.php?show=search_by_id&id=elxkUBQ%3D&keyword=klasa

Imenovanje klasa mora biti smisleno jer će u protivnom održavanje rječnika biti noćna mora. Jednom kada broj klasā i riječi naraste, bit će nemoguće pamtiti koja je klasa za što, zato treba raditi na sljedeći način: sve imeničke klase počinju slovom A i ovisno o karakteristikama deklinacije dodaje se još jedno slovo, recimo u slučaju imenica muškoga roda bez glasovnih promjena doda je se još jedno A, dakle klasa je AA. U slučaju imenica muškoga roda s nepostojanim *a*, ali ovim ili onim nastavkom u vokativu obrazac B, dakle imenica s obrascem B nosi oznaku AB. U slučaju palatalizacije te ovog ili onog nastavka za instrumental dodaje se klasa C, dakle AC i tako redom kako se na situacije prilikom iščitavanja rječnika nailazi. Treba voditi računa o tome da se neke riječi pojavljuju samo u jednini, a neke samo u množini. Treba voditi računa da neke riječi imaju više oblika u genitivu množine: *bajki*, *bajaka* i *bajkā* (bajka, kratka priča). Također će trebati posebna imenička klasa ako postoji mogućnost dvojnih nastavaka za instrumental jednine imenica muškog roda na -ar: *mornarom* i *mornarem*. Da se ne ulazi previše u gramatiku hrvatskoga jezika jer bi to izašlo iz okvira ovoga dokumenta, treba samo reći da su nazivi klasa proizvoljni, ali olakšat će održavanje ako su smisleni. Korisno je voditi tekstnu datoteku s popisima klasa i što rade radi dokumentiranja samoga rječnika, ali i činjenice da će takav popis olakšati dodavanje novih riječi. Trenutačno takav popis ne postoji, ali kao rješenje toga problema treba spomenuti ProofingToolGUI[8], alat koji omogućuje uređivanje pravila za računalnu provjeru riječi Hunspellom.

I naravno, jednoj riječi moguće je dodijeliti više klasa; u primjeru ispod riječi *mornar* dodijeljene su tri dvoslovne klase: AA, AB te AD.

```
1
mornar/AAABAD
```

Trenutačni rječnik koristi dvoslovne klase, ali ne prati predložen sustav jer je rječnik rastao organski. Problem je što trenutačna datoteka afiksā razrađuje i tvorbu što je zapravo pogrešno jer nevjerojatno otežava održavanje rječnika, a upravo takav sustav funkcionira dodjeljivanjem više klasa jednoj riječi, kao što se vidi u gornjem primjeru. Više o tome u dijelu `Tvorba naspram dodavanja samo gramatičkih nastavaka.`

U rječničku datoteku moguće je unijeti i dvije riječi u jedan redak, ali nije moguće obje mijenjati. Ova je značajka dobra za `New York` jer se prva riječ ne sklanja:

```
1
New York/AG
```

Međutim nije moguće pisati `Crna/CA Gora/AB` jer Hunspell ne može uparivati deklinaciju prve i druge riječi i sinkronizirano davati izmjene za zatipak. Hunspell provjerava postojanje klase samo na kraju unosa. Iako je moguće, radi sustavnosti ovakve unose treba izbjegavati.

Naravno, nesklonjive riječi se unose u rječnik bez ikakvih klasa; vidi riječ *ujutro* u primjeru:

```
4
rječnik/AA
trčati/BA
```

[8] https://proofingtoolgui.org/

```
ujutro
žena/AB
```

Važno je voditi računa o tome da se riječi dodaju u rječnik s velikim slovom ako je riječ o vlastitim imenima. Uvrsti li se u rječnik riječ `hrvatska` (malim slovom), rječnik će dozvoljavati i *hrvatska* i *Hrvatska*, no ako se unese velikim slovom, prihvaćat će samo *Hrvatska*, što je dvosjekli mač jer korisnik može napisati i *hrvatska kultura*, a tu treba ići malo slovo. U tom se slučaju radi sustavnosti u rječniku trebaju nalaziti i *Hrvatska* i *hrvatska*. Naravno, spomenuto će vrijediti i za *Crna Gora* jer u rječniku moraju biti riječi *crna* i *gora*. U prvom je slučaju u kontekstu vlastita imena, u drugom u kontekstu opće imenice i pridjeva. Ovo postavlja temelj ako će se danas-sutra rječnik označavati (engl. *POS tagging*) jer se imenice mogu označiti i kao opće i kao vlastite. Time se osigurava da kakav jezični model razumije je li riječ o općoj vezi ili vlastitom imenu. Isto je i s vlastitim imenima nastalima od opće imenice:

```
2
višnja/AB
Višnja/AB
```

O tome što i kako označavati u ovome dijelu teksta neće biti govora jer je riječ o vrlo složenom problemu kojemu treba pristupiti oprezno jer upravo su oznake ono što umjetna inteligencija i napredna provjera gramatike, tipografije i stila koriste za ispravljanje pogrešaka. Dotaknut će se toga u nekom od sljedećih poglavlja, no evo primjera označene rječničke datoteke:

```
4
rječnik/AA  st:rječnik  po:imenica is:muški is:neživo is:jednina is:nominativ
trčati/BA   st:trčati   po:glagol  is:infinitiv is:nesvršeni
ujutro      st:ujutro   po:prilog  is:vremenski
žena/AB     st:žena     po:imenica is:ženski   is:jednina  is:nominativ
```

U ovome trenutku napisano nije prijedlog označavanje rječnika jer će o tome biti spomena u poglavlju `Označavanje rječnika za naprednu provjeru gramatike`; no reći će se da je važno odvojiti oznake tabulatorom te ispred vrijednosti staviti troznakovni odjelitelj. Tako `st:` znači `stemm`, odnosno predstavlja kanonski rječnički oblik, što je korisno za supletivne osnove (čovjek ~ ljudi) ili homonimske parove: `pomoći` kao glagol u infinitivu naspram `pomoći` kao Gjd. imenice ženskog roda od riječi *pomoć*. Oznaka `po:` predstavlja (engl.) *part-of-speech*, odnosno sadrži vrstu riječi. Posljednja oznaka u ovom primjeru je `is:` koja znači (engl.) *inflectional suffix* i guta sva značenja gramatičkoga morfema. Troznakovnih označitelja ima još, ali reći će se još samo da LightProof, LibreOfficeova ugrađena provjera gramatike, koristi ovakve oznake u hrvatskom rječniku kako bi razlikovala infinitive od ostalih riječi. Bez toga provjeravati krnji infinitiv u futuru ne bi bilo moguće. Ne samo to nego označen rječnik može poslužiti i za morfološku analizu rečenice, a što je značajka koja je u Hunspellu podržana. Dakle označavanje nije samo sebi svrhom, nego ima praktičnu primjenu i zapravo je nužnost.

Datoteka afiksā – logika iza rječnika

Budući da je rječnička datoteka jednostavan popis riječi, datoteka afiksā je logika koja od popisa kanonskih oblika riječi stvara cijele paradigme. Od jednoga unosa u prvoj druga u nekim slučajevima stvara desetke različitih što oblika što novih riječi. Kada se jednom shvati, sintaksa pravila i nije toliko složena.

Prefiksi (PFX), sufiksi (SFX) i uvjeti u klasama

Ako se prisjeti da se u rječničku datoteku upisalo `rječnik/AA`, onda u datoteci afiksā u jednom trenutku treba očekivati

```
SFX   AA    Y     13
SFX   AA    0     a     .
SFX   AA    0     u     .
SFX   AA    0     0     .
SFX   AA    k     če    k
SFX   AA    0     u     .
SFX   AA    0     om    .
SFX   AA    k     ci    k
SFX   AA    0     a     .
SFX   AA    k     cima  .
SFX   AA    0     e     .
SFX   AA    0     ci    .
SFX   AA    k     cima  .
SFX   AA    k     cima  .
```

Krenut će se od prvoga retka, koji je zapravo zaglavni redak i ne sudjeluje u stvaranju novih oblika riječi, već deklarira sljedeće: `SFX` znači *sufiks*, dakle pravila se primjenjuju na kraj riječi. Oznaka, odnosno naziv klase je, kao što je već rečeno, dvoslovna oznaka AA. Broj trinaest označava koliko je ukupno pravila u klasi. Možda se na prvu trinaest može činiti čudnim brojem, ali nije: sedam padeža u jednini i sedam u množini daju četrnaest – prvo je (ili četrnaesto, ovisno kako se gleda) već u rječničkoj datoteci. Brojevi, stoga, odgovaraju.

Sljedeći redak ponavlja podatak da je riječ o sufiksu naziva klase AA, a nakon slijedi znak koji treba s kraja riječi `ukloniti`, nakon čega slijedi znak/slovo koje treba `dodati` i posljednji znak u primjeru označava `uvjet` pod kojim se uklanjanje i dodavanje znaka može dogoditi. Dakle napisano riječima, mogli bismo prikazati i ovako:

Sufiks	naziv klase	ukloniti znak	dodati znak	uvjet
SFX	AA	0	a	.

Ovdje je sada nužno dotaći se i gramatike, naime riječ `rječnik` je imenica muškoga roda koja u nominativu jednine ima nulti gramatički nastavak: dakle `rječnik` + ø. Budući da u genitivu jednine samo dodajemo nastavak `-a` nema potrebe uklanjati ništa s kraja riječi i otuda ova `0` (riječima: nula). Sve se to treba dogoditi bez obzira na koje glasove ili slova riječ završava. To

označava `točka`. No umjesto točke na tom mjestu može stajati i kakav kompleksniji uvjet. Evo primjera za vokativ jednine imenice *rječnik*:

Sufiks	naziv klase	ukloniti znak	dodati znak	uvjet
SFX	AA	k	če	k

U ovom slučaju sufiks naziva AA uklanja `-k` s kraja riječi i na ono što ostane lijepi `-če`, ali samo ako riječ završava na `-k`. Evo kako bi se to moglo rastaviti na morfeme: Njd. `rječ/nik/ø`, Gjd. `rječ/nik/a` te Vjd. `rječ/nič/e`. Zašto se prikazuju morfemi? Zato što tehnički Hunspell treba nalijepiti `-e` na `rječnik` i time se provodi palatalizacija: glas *e* iza *k, g* i *h* daje *č, ž* i *š*. No kako se u ovom slučaju moramo odvojiti od morfologije kako je opisuje gramatika jer ne možemo reći Hunspellu dodaj samo nastavak `-e` i automatski pretvori `-k` u `-č`, nego doslovno treba zamijeniti `-k-` sa `-č-`.U tom slučaju se, kao što je spomenuto, odmičemo od morfologije kako je opisna u gramatikama, ali stvar je tehničkoga ograničenja. Pratiti morfeme u gramatičkom (priručničkom) smislu nije uvijek moguće. Može se oponašati do neke mjere, ali iznimke će se pojavljivati.

Evo još jednog primjera sa složenijim uvjetom iz stvarnoga rječnika za provjeru riječi:

Sufiks	naziv klase	ukloniti znak	dodati znak	uvjet
SFX	VO	0	i	[^kgh]

U ovom slučaju sufiks klase naziva V0 ne uklanja ništa s kraja riječi nego dodaje slovo `-i` ako riječ ne `završava` na k, g ili h. Ovo bi znači da se sibilarizacija u konkretnom slučaju ne provodi; to znači znak ^ kada je unutar uglatih zagrada. Slova unutar tih zgrada znače da uvjet traži da riječ ne završava ni na jedno od tih slova. Evo suprotnoga primjera:

Sufiks	naziv klase	ukloniti znak	dodati znak	uvjet
SFX	AA	k	ci	[kgh]

Sada se uklanja završno `-k` i dodaje `-ci`, recimo unuk — `unuci`.

Međutim uvjet može biti još složeniji ili još jednostavniji. Ako pojednostavimo i zanemarimo zagrade, možemo za `točka — točki` napisati

Sufiks	naziv klase	ukloniti znak	dodati znak	uvjet
SFX	AA	a	i	čka

zato što u imenicama ženskoga roda na `-cka`, `-čka`, `-ćka` i `-tka` i `-zga` ne provodi sibilarizacija: `kocki`, `točki`, `tvrtki`, `mazgi`. Budući da ovi sufiksi završavaju na `-ka` i razlikuju se samo u slovu ispred, moguće je uvjet postaviti i ovako `[cčćtz]ka`. Dakle uvjet je ispunjen ako riječ završava na jedno od slova iz uglate zagrade plus (fiksno) `-ka`. Uvjeti su moćna stvar i sadašnji rječnik to (zlo)upotrebljava: što je više paradigmi s takvim kompliciranim uvjetima to je veća vjerojatnost da se takva klasa doda riječi jer se misli da je time sve pokriveno, ali riječ je baš o iznimci i onda rječnik generira nepostojeće oblike. Vidi više u `Tvorba naspram dodavanja samo gramatičkih nastavaka`. Ako se rade prefiksi jednostavno se `SFX` zamijeni s `PFX`. Sve ostalo ostaje isto. Što ako istovremeno želimo dodati i prefiks i sufiks kao u slučaju komparativa i superlativa za pridjeve?

hr_HR.dic

```
1
zelen/CACB
```

hr_HR.aff

```
SFX    CA     Y      1
SFX    CA     0      iji      .

PFX    CB     Y      1
PFX    CB     0      naj
```

Ovo će generirati i `zelen` (jer je tako u rječničkoj datoteci) i `zeleniji` (dodan sufiks –iji) i `najzeleniji` (dodan i prefiks i sufiks).

Crossproduct – istovremeno kombiniranje sufiksa i prefiksa

Razlog zašto se mogu istovremeno dodati i prefiks i sufiks je u ovom Y (za C*rossproduct Yes*). Kada bi umjesto Y stajalo N (`SFX CA N 1`), to pravilo se ne bi moglo istovremeno kombinirati s prefiksom i sufiksom. Moglo bi biti `zelen`, `zeleniji` i `najzelen`, ali ne i `najzeleniji`.

Međutim, rezultat postavljanja datoteka kao u primjeru bit će i `najzelen`. Sadašnji rječnik tome problemu doskače na način da jednostavno u rječničku datoteku uvodi i pozitiv i komparativ:

```
2
zelen/CA
zeleniji/CB
```

te onda iz komparativa tvori superlativ, ali to je nepotrebno. Ovaj se problem može riješiti dodavanjem direktive NEEDAFFIX ZZ na vrh datoteke.

NEEDAFFIX – prijedlog za komparativ i superlativ

Doduše ne na sam vrh, ali o tom kasnije. Datoteku afiksā počelo se objašnjavati od kraja prema početku da se odmah poveže s rječničkom datotekom. Pravila za afikse idu na dno, a iznad njih idu konfiguracijske opcije i direktive. Evo kako bi to trebalo izgledati za sada:

hr_HR.dic

```
 1
zelen/CACB
```

hr_HR.aff

```
NEEDAFFIX  ZZ

SFX    CA     Y      2
SFX    CA     0      iji             .
```

```
PFX     CB      Y       1
PFX     CB      0       naj/ZZ          .
```

U prvom takvom retku dozvoljavamo `zeleniji`, a u drugom retku dozvoljavamo `iji`, ali u kombinaciji s klasom `CB`. Kako ta klasa ima oznaku `ZZ` (`naj/ZZ`), ne može stajati samostalno. Klasa CB može stajati na rječničkom unosu samo ako je riječi dodana još jedna klasa afiksā. Na ovaj način bi se iz rječničke datoteke izbacili komparativi i prebacili na/u datoteku afiksā.

Kompresija klasa pomoću direktive AF

Razlog zašto je bilo potrebno razvijati vlastiti alat `dictman.php`[9] jest činjenica da ugrađeni alat `unmunch`[10] za generiranje svih oblika svih riječi u rječniku ne radi s direktivom AF. Hrvatski rječnik koristi ovu značajku jer sadržava velik broj klasa na jednoj riječi, što onemogućava kvalitetno održavanje rječnika. Neka se pretpostavi da riječ *računalo* ima pet klasa: `računalo/AAABACADAE`; na više od pedeset tisuća unosa u rječničku datoteku pohvatati kojoj riječi treba dodati koje klase ili zastavice, jednostavno nije moguće. Dobra stvar je što se u predloženome ovih pet zastavica upisuje samo jednom, no do problema dolazi kada treba dodati kakvu imenicu od koje se, osim deklinacije, radi i pridjev, zato se u datoteku afiksā unosi sjedeće:

`hr_HR.aff`

```
AF 1
AF AAABACADAE       # 1
```

U ovom minimalnom primjeru svih pet klasa mijenja se (zamjenjuje) brojem `1`. Prednosti nisu samo praktične prirode u smislu održavanja rječnika, već postoje i ozbiljne prednosti u vidu performansi same provjere, brže učitavanje rječnika u radnu memoriju računala, ali i smanjenjem diskovnoga prostora. Sve iza znaka # predstavlja komentar održavatelja tako da je iza njega moguće dodatno opisati što koja skupina klasa radi

```
AF 2
AF AAABACADAE       # 1
AF AAABAD           # 2     Imenice m. r., -u u vokativu, dvojni instrumental (-om/-em), pridjev na -ski
```

Ako se datoteka afiksā ovako postavi, u rječničku datoteku više se ne upisuju klase ili zastavice, već broj mjesta pojavljivanja (indeks) unutar direktive AF.

`hr_HR.dic`

```
2
računalo/1
mornar/2
```

[9] https://github.com/krunose/hunspell-hr/tree/master/tools
[10] Jedan od alata unutar GNU/Linux paketa `husnpell-tools`

Međutim, ovakav pristup ima i potencijalni problem: nove klase se mogu dodavati samo na kraj direktive jer je pozicija skupine klasa ili zastavica unutar nje fiksna i određena njezinim mjestom pojavljivanja u tablici. Ako riječ *mornar* ima oznaku `/2` u rječničkoj datoteci, kada se između nje i riječi `računalo/1` ugura još jedan alijas, *mornar* postaje `/3`, a u rječničkoj datoteci i dalje ima oznaku `/2`. U tom slučaju je potrebno ispravljati alijase svim takvim riječima u rječničkoj datoteci. Bolje je novi set klasa i zastavica dodati na kraj direktive i izmijeniti broj na njezinu vrhu.

Zbog činjenice da su alijasi uvjetovani mjestom pojavljivanja unutar direktive, dodjeljivati ih se ne treba tijekom izrade rječnika (ako bi se rječnik radio ispočetka), već se to napravilo onda kada je rječnik bio gotov. Računalnom skriptom su se učitale klase i zastavice na svim riječima unutar rječničke datoteke, filtrirali ponavljajući unosi i generirala tablica redom pojavljivanja u rječniku. Ako se zna da sadašnji rječnik ima preko 400 takvih kombinacija, jasno je da nije moguće brojati svaki put kada se želi dodati koja riječ u rječnik, zato su uvedeni komentari s brojevima. Izvorno rječnik nije koristio alijase, nego se niz od više klasa nalazio u rječničkoj datoteci, no shvatilo se da tako neće ići.

Tablica sustavnih pogrešaka: direktiva REP

Ono što nam od dna prema vrhu slijedi je REP (*Replacements*). Računalni provjernik riječi može predvidjeti najčešće pogreške pri pisanju riječi i automatski te pogreške zamijeniti ispravcima kako bi nudio (puno) bolje prijedloge za ispravak. Struktura sekcije REP slična je onoj sekcije AF.

```
REP 1
REP c ds
```

Ono što treba primijetiti jest ključna riječ REP te (opet) broj takvih pravila u datoteci nanizanih jedno za drugim, ali već u sljedećem retku postoji novo: provjerava se je li u riječi slovo `c`, ako jest, zamijeni ga skupinom `ds` i provjeri postoji li sada riječ u rječniku: `pre`**`c`**`ijednik` > `pre`**`d-s`**`ijednik` > `predsjednik`.

```
NEEDAFFIX ZZ


REP 1
REP c dc


AF 1
AF AAABAD   # 1

[ . . . ]
```

Međutim ova sekcija ima jednu specifičnost – izvršava se od vrha prema dnu i Hunspell prestaje provjeravati prvi put kada naiđe na pogodak. To samo po sebi nije ni dobro ni loše, ali pravila treba pisati od konkretnoga i specifičnoga prema općem: treba uzeti okolinu unosa ako je moguće kako bi sekcija pogodila baš riječ za koju se dodao konkretan REP. Ponekad se dogodi da

je točan prijedlog tek treći ili četvrti u prijedlozima jer je korisnička greška sličnija kojoj drugoj riječi u rječniku nego onome što je stvarno mislio napisati, zato REP rješava česte greške u pisanju i na taj način nudi puno bolje zamjene za korisničke pogreške.

U ovoj sekciji moguće je naznačavati da riječ počinje određenim nizom znakova i da završava određenim nizom znakova:

```
REP 4
REP ^zdrastven zdravstven
REP ^videe$ videa
REP ićeš$ it_ćeš
REP c dc
```

Kao što je rečeno, specifično (i dugačko) na vrh, a općenito (i kratko) na dno. Znak ^ označava da riječ mora početi određenim nizom znakova, a znak $ da riječ završava određenim nizom znakova. Naravno, moguće su i kombinacije. Evo što konkretno rješava treći redak REP sekcije: `ićeš$ > it_ćeš` (< kupićeš < kupit ćeš). Razmak se označava znakom _. Naravno, u ovom dijelu se rješava i pitanje pisanja *ije* i *je* te *č* i *ć*, ali i *dž* i *đ*; ali možemo rješavati i pisanje niječne čestice zajedno s glagolom: `REP ^ne ne_`: `neznam > ne znam`; `nemogu > ne mogu`.

```
REP 9
REP ^zdrastven zdravstven
REP ^videe$ videa
REP ićeš$ it_ćeš
REP ije je
REP je ije
REP dž đ
REP đ dž
REP c dc
REP ^ne ne_
```

Ovaj dio je skupljan godinama i vjerojatno sadrži pogreške, duple unose i moguće ga je optimizirati, ali funkcionira, a ne znam tko će ga prikupiti ispočetka. Bolje ga je samo dopunjavati.

Higijena ulaznih znakova: tablica ICONV

Sljedeća direktiva je `ICONV`. U sadašnjem rječniku se koristi da bi se riječi s naglascima ponašale kao riječi bez naglasaka, tako riječ `sâm` Hunspell vidi kao `sam` (bez naglaska) i ne podcrtava je.

```
ICONV 32
ICONV à a
ICONV â a
ICONV â a
ICONV à a
[ . . . ]


REP 9
```

```
REP ^zdrastven zdravstven
REP ^videe$ videa
REP ićeš$ it_ćeš
[ . . . ]
```

Ova značajka je u pravilima za provjeru riječi u hrvatskom dobro došla zbog naglasaka, ali povijesno je služila vremenu kada nije sve bilo standardizirano u UTF-8. Računala su mogla, recimo slovo *č* zapisati kao jedan znak s kvačicom, ili kao dva znaka: slovo *c* na koji se nadoveže kvačica koja vizualno izgleda kao jedno slovo, ali tehnički je riječ o dva znaka. ICONV je čistio i standardizirao na razini rječnika takve situacije.

Ne samo znakove s kvačicama, ICONV može čistiti i znakove poput kosoga apostrofa (`) budući da u hrvatskome koristimo ravan ('), pa se može dodati redak

```
ICONV ``'
```

čime će Hunspell zamijeniti navodnike i, recimo ako je tekst kopiran s interneta, nastaviti provjeravati ispravnost riječi bez prekida ili nepotrebnoga podcrtavanja. Bitno je zbog riječi koje se pišu s apostrofom: *McDonald's*.

BREAK i WORDCHARS – rastavljanje riječi na određenom (pravopisnom) znaku

Jasno je da u jeziku riječ nije samo niz slovnih znakova, već se mogu pojaviti i drugi (posebni) znakovi. Direktiva BREAK rastavlja riječ na tom znaku nadvoje i svaki dio pregledava zasebno.

```
BREAK 5
BREAK ^-
BREAK -$
BREAK ^'
BREAK '$
BREAK .
```

Prije negoli se objasni ovaj dio rječnika, treba spomenuti da BREAK funkcionira ruku pod ruku s direktivom WORDCHARS; krenimo od nje: riječ ne čini samo niz slovnih znakova, već se unutar riječi mogu pojaviti i posebni znakovi: *spomen-ploča*, *remek-djelo*. Hunspell će provjeriti postoji li u rječniku *spomen* i postoji li *ploča*. Svaki dio polusloženice provjeravat će zasebno. Izgleda dobro, ali što kada se pojave polusloženice u kojima jedna od sastavnica ne može stajati samostalno: *remek-djelo*. Ne postoji riječ *remek* pa je pogrešno dodavati je u rječnik, zato se polusloženice u rječnik unose u cijelosti, zajedno sa spojnicom. Da bi provjernik riječi prepoznao spojnicu kao sastavni dio riječi, treba dodati direktivu WORDCHARS.

```
WORDCHARS -'0123456789.
```

Direktiva je to koja će gore navedene znakove tretirati kao sastavni dio riječi. U tom slučaju se u rječnik doslovno unosi `spomen-ploča` i `remek-djelo`, a spojnica postaje njezin dio. Prije unosa direktive WORDCHARS provjernik nije davao dobre prijedloge za polusloženice i uporno je pod-

crtavao samo *remek* u polusloženici *remek-djelo*. Apostrof je označen kao sastavni dio riječi zbog *McDonald's* i sličnih naziva stranih robnih marki.

Ako spojnicu treba tretirati kao sastavni dio riječi u sredini polusloženice, svakako spojnicu ne treba gledati kao sastavni dio riječi na početku i kraju, recimo, gramatičkih i tvorbenih nastavaka: `-čka`, `naj-` i slično. Zato imamo `BREAK ^-` i `BREAK -$` (ne tretiraj spojnicu kao dio riječi ako je na početku, odnosno kraju riječi). Isto je s apostrofom.

Problem je što u datoteci `hr_HR.aff` postoji `BREAK .` i `WORDCHARS .` Sada su WORDCHARS i BREAK u koliziji. Prva direktiva traži da točka bude dio riječi, a druga direktiva traži da se spojene riječi rastave na dvije sastavnice: jedna lijevo od točke, druga desno. Razlog zašto se točka uvela u direktivu `WORDCHARS` je to što se pokušalo ispravljati zatipke u pisanju skraćenica. Logika je bila da se točka tretira kao sastavni dio riječi i da se skraćenice u rječnik unose s točkom pa će u slučaju zatipka sustav znati dati dobar ispravak (zajedno s točkom). Može doći do ponavljanja točke ako je kratica ujedno i kraj rečenice: [...] *imenice, pridjevi idr.* > [...] *imenice, pridjevi i dr.*. Kako su ove direktive sada napisane, može doći do takve kolizije jer sustav pogrešku (*idr.*) ispravlja, ali nudi i točku jer je kratica s točkom unesena u rječnik, a točka je već stavljena za kraj rečenice. Ovo nije problem Hunspella, nego njegove implementacije u pojedinu aplikaciju.

Nedostatak je primijećen na istom rječniku korištenom u dva različita obrađivača teksta otvorenoga koda. Jedan je LibreOffice, a drugi je OnlyOffice. Ovo možda nije nužno problem Hunspella koliko je pitanje načina na koji pojedini obrađivač teksta rastavljaju tekst na rečenice pa onda na riječi i u kojem ih obliku prosljeđuju provjerniku. Problem leži u tome što svaki obrađivač teksta (engl. *word processor*) rastavlja rečenice po svojim pravilima. Isti tekst u LibreOfficeu i OnlyOfficeu neće jednako biti rastavljen na rečenice. LibreOffice će znati da kratica *dr.* ne predstavlja kraj rečenice jer mu se eksplicitno zadalo da točka u tom slučaju nije kraj. OnlyOffice nema tu značajku; svaku točku iza koje slijedi razmak tretira kao kraj rečenice.

Sada se postavlja pitanja prema kojoj aplikaciji raditi i kalibrirati pravila. Mora se donijeti zaključak da je jedini ispravni put pratiti ono što radi Hunspell u naredbenom retku. Dakle ne može se ispravljati pravila na jednom rječniku za svaku implementaciju, potrebno je raditi tako da radi unutar samoga Hunspella, a sve ostale pogrešne implementacije treba prijaviti kao (ozbiljan) nedostatak (engl. *bug*) aplikaciji koja implementira značajke Hunspella neispravno.

FLAG long – smisleni nazivi klasa

U prvim poglavljima sugeriralo se dodavanje dvoslovnih klasa/zastavica, ali prema zadanim postavkama Hunspell za nazive klasa koristi samo jedno slovo ili, umjesto slova, brojeve. Slova se u rječničkoj datoteci samo nižu jedno za drugim, no brojevi se moraju odvajati zarezom. Da bi se dobio dvoslovni sustav klasa, trebalo je uvesti direktivu `FLAG long`.

```
FLAG long

FULLSTRIP

BREAK 5
BREAK ^-
```

```
BREAK -$
BREAK ^'
BREAK '$
BREAK .

WORDCHARS -'0123456789.
```

Nažalost, trenutačno rječnik ne prati predloženu sistematizaciju pri formiranju naziva za klase nego ide redom, od kraja (engleske) abecede prema početku. To nije nužno loše jer se sada intervencije u rječniku mogu napraviti upravo tako da se krenu čistiti, recimo, glagoli i njima se dodijeli naziv klase kojima će prvo slovo biti A, a zatim ovisno o glagolskoj vrsti dodaje se još jedno slovo. U rubnim slučajevima i iznimkama dodaje se još sljedeće slovo i tako redom.

Abecede ima dovoljno jer ako postoji deset vrsta riječi, s time da nepromjenjivim vrstama riječi ne treba dodavati klase, onda ima i više nego dovoljno da se sve situacije pokriju s 26 x 26 (676 kombinacija) slova engleske abecede. Ako se kreće od glagola – a trebalo bi zbog provjere gramatike LightProofom i činjenice da je na njima već odrađen dio označavanja rječnika, a i najčišći su za rastaviti – onda neka njihove dvoslovne klase počinju s A. Sljedeća su skupina koju bi trebalo rješavati pridjevi. Ima ih najviše, a razradilo se pitanje stupnjevanje pridjeva. Njihovi deklinacijski obrasci su relativno jednostavni i nemaju toliko glasovnih promjena.

FULLSTRIP – zamjena cijelih riječi iz rječnika

Ako se u drugom koraku ide na pridjeve, onda treba voditi računa o tome da postoje i pridjevi kojima se mijenja cijela osnova: *dobar*, *zao*, *velik*, *malen*. Da bi se od *dobar* dobilo *bolji* (potom i *najbolji*), datoteka afiksā u svojim pravilima mora ukloniti cijelu riječ koja stoji u rječničkoj datoteci. Bez direktive `FULLSTRIP` to nije moguće. Tradicionalno pri stvaranju takvih klasa, afiksi nisu smjeli ukloniti cijelu riječ, trebalo je ostati barem jedno slovo. Ova direktiva rješava taj problem.

`hr_HR.dic`

```
1
dobar/BA
```

`hr_HR.aff`

```
FULLSTRIP

[ . . . ]

SFX   BA    Y     1
SFX   BA    dobar bolji dobar
```

Nije tu samo pet nepravilnih pridjeva, tu su još i zamjenice poput *ja - mene*, *on - njega*, *ona - nje/je/ju*... Tu je i imenica *čovjek – ljudi.* FULLSTRIP je nužnost radi sustavnosti rječnika.

SET, TRY, KEY – samo ukratko

Kako smo datoteku afiksā počeli opisivati od kraja prema početku, da se odmah poveže i nadoveže na ono što stoji u rječničkoj datoteci, sada se došlo do samoga vrha, do onoga što je zapravo vrlo bitno, ali ne mijenja oblike riječi; no nužna je tehnikalija.

```
 SET UTF-8

TRY aionerjmtsulkpvdzAgčcbšhćžfEMđGCILHDJSKPBŠTVČNRFOZŽWyUĐxwYĆQqX

KEY qwertzuiopšđ|asdfghjklčćž|yxcvbnm|žšđ|ćščđž|žćđ|đšćž|zy|qawsedrft-
gyhuj|jikolpč|aysxdcfvgbh|hnjmk
```

Direktiva `SET UTF-8` proglašava u kojem je standardu kodiran cjelokupni tekst rječničke datoteke i datoteke afiksā. Ranije su to bili različiti standardi poput `Windows-1250` ili `ISO-8859-2`. Konvertiranjem rječničke i datoteke afiksā u standard `UTF-8` i deklariranjem toga na samome vrhu, postavljen je univerzalni temelj i osiguralo se da ne dolazi do problema s konverzijama.

Direktiva TRY je iznimno bitna u trenutku kada korisnik napravi zatipak. Poredak slova i znakova u ovome dijelu razlikuje se od abecednoga jer dolazi isključivo prema statističkoj učestalosti pojavljivanja slova u hrvatskom jeziku. Hunspell će pri računanju Levenshteinove udaljenosti (matematičke sličnosti među riječima) prioritet u pogađanju dati slovima s početka niza.

Dok se TRY muči sa statističkim rasporedom i učestalošću pojavljivanja slova, direktiva KEY čini to isto samo u fizičkom smislu: predstavlja raspored slova na hrvatskoj tipkovnici. Uspravnim crtama odvojeni su znakovi koji se na hrvatskoj tipkovnici (QWERTZ) nalaze jedan pored drugoga. Zatipci su vrlo česti zbog slučajnoga pritiska krive tipke; direktiva KEY definira koje su tipke jedna pored druge i prvo pretpostavlja da je zatipak nastao iz toga razloga.

Prijedlog rješenja za brojeve uz imenicu ili pridjev

Ako se pogleda datoteka afiksā sadašnjega rječnika[11],vidjet će se cijeli niz direktiva koje još nisu opisane, a nisu ni implementirane. Unesene su u datoteku afiksā samo radi čuvanja slovne oznake, odnosno zastavice (sada ipak *zastavice* jer je u ovom slučaju teže govoriti o klasama). Neke su od njih potrebne za rješavanje pitanja s kojima se muku muči već neko vrijeme: kako Hunspellom riješiti pisanje složenica od kojih je prva sastavnica broj, a druga imenica ili pridjev: 20-godišnji, 22-godišnjak... Brojeva je bezbroj i nije ih moguće sve popisati i unijeti u rječnik. Oni u rječnik ne moraju biti upisani samo brojkom, ispravne riječi su i *dvotjedni*, *dvanaestomjesečni*, ali i *pedesetpetogodišnjak* i *tridesetogodišnjakinja*. Usredotočimo se na trenutak na složenice sastavljene od znamenki te imenice ili pridjeva.

Značajka Hunspella koja može riješiti ovo pitanje jest COMPOUND. Budući da je Hunspell izvorno pisan s mađarskim na pameti, a to je aglutinativni jezik, ova značajka dozvoljava lijepljenje jedne riječi ili sufiksa iz rječnika na drugu takvu riječ (ili sufiks/prefiks) čime se dobiva treća riječ,

[11] https://github.com/krunose/hunspell-hr/blob/master/hr_HR.aff

koja samostalno nije upisana u rječnik. Pretpostavi li se unos u rječniku `dvo` i `tjedni`, od ove dvije riječi moguće je dobiti `dvotjedni`, koji se, zapravo, ne nalazi u rječniku. Neka nikoga ne muči unositi u rječnik oblik *dvo* jer on samostalno neće proći kao točan ako mu se dodijeli zastavica za `ONLYINCOMPOUND`, odnosno može se pojaviti isključivo kao dio složenice, i to je u ovom slučaju točno. Tako se od niza znamenki od 0 do 9 i imenice *godišnjak* mogu dobiti svi brojevi povezani spojnim elementom (spojnicom) sa spomenutom imenicom: 22-godišnjak, 23-godišnjak, 24-godišnjak... U nastavku se donosi minimalan takav primjer.

`hr_HR.aff`

```
[ . . . ]

COMPOUNDMIN 1

WORDCHARS -'0123456789.

ONLYINCOMPOUND ZW
COMPOUNDBEGIN ZU
COMPOUNDEND ZS

COMPOUNDRULE 1
COMPOUNDRULE (ZU)*(ZS)

REP 1
REP god -god

SFX AA Y 5
SFX AA i eg i
SFX AA i ega i
SFX AA i em i
SFX AA i emu i
SFX AA i im i
```

`hr_HR.dic`

```
11
0/ZU
1/ZU
2/ZU
3/ZU
4/ZU
5/ZU
6/ZU
7/ZU
8/ZU
9/ZU
-godišnji/AAZSZW
```

Vjerojatno je u ovom trenutku već moguće razumjeti što se u datoteci događa i bez posebnoga objašnjavanja, ali brojevni niz u WORDCHARS je vrlo važan. Naime, ako Hunspell vidi brojeve od nula do devet kao što gleda na slova abecede, onda je svaki broj zapravo riječ koju treba dodati u rječničku datoteku. Naravno, u rječničku datoteku dodao se i znakovni niz `-godišnji` jer je jednostavnije unijeti tako u rječnik i dodati zastavicu ONLYINCOMPOUND, čime je takav oblik moguće dobiti samo unutar složenice, a ne i izvan nje. Tako su označeni brojevi od 0 do 9 u rječničkoj datoteci. Direktiva COMPOUNDEND označava posljednji dio složenice.

Treba još spomenuti COMPOUNDRULE direktivu koja ima (za sada) dva retka: prvi je deklaracija koliko pravila slijedi (u ovom slučaju jedno) i drugi redak predstavlja obrazac pravila za spajanje složenica. Budući da smo brojeve označili s COMPOUNDBEGIN ZU, iza te zastavice treba dodati zvjezdicu jer ona označava mogućnost pojavljivanja nula ili više znamenki jedne do druge; zatim je dodana zastavica ZS kako bi Hunspell prepoznao što treba biti posljednji dio sastavnice. COMPOUNDMIN definira minimalnu duljinu (broj slova ili znamenki) pojedine sastavnice u složenicama. Upisalo se 1 kako bi se pokrili i jednoznamenkasti brojevi.

Problem još nije riješen: što ako umjesto `22-godišnji` piše `dvadesetdvogodišnji` ili `tridesetdvogodišnjak`. Sada imamo prvu sastavnicu složenica napisanu slovima, a sama može biti složena od više dijelova: (dv) + (o) + (godišnji), (dva + deset) + (o) + (godišnji), (sto + tri + deset + dv) + (o) + (godišnji)... COMPOUNDEND smo potrošili na `-godišnji` i ne možemo istu klasu sada dodijeliti na `godišnji` u `dv + o + godišnji` jer bi provjernik počeo propuštati situacije kao `22godišnjak`, a ne može se dopustiti da se točnim proglašava ono što točno nije. Ako bi COMPOUNDBEGIN ZU dodali još i na `dvo-`, `tro-`, `dvadesetdvo-`, mogućim bi postali (u teoriji, nije isprobano) i nepostojeći oblici poput `dvo-godišnji` jer ostaje problem s COMPOUNDEND. To sustav ne smije propustiti i dopustiti. Evo prijedloga kako to riješiti:

`hr_HR.aff`

```
[ . . . ]

CHECKCOMPOUNDPATTERN 1
CHECKCOMPOUNDPATTERN o o [o]

COMPOUNDRULE 8
COMPOUNDRULE (ZU)*(ZS)
COMPOUNDRULE (ZN)(ZO)(ZP)
COMPOUNDRULE (ZM)(ZO)(ZP)
COMPOUNDRULE (ZM)(ZN)(ZO)(ZP)
COMPOUNDRULE (ZL)(ZM)(ZN)(ZO)(ZP)
COMPOUNDRULE (ZK)(ZL)(ZP)
COMPOUNDRULE (ZL)(ZP)
COMPOUNDRULE (ZK)(ZL)(ZM)(ZN)(ZO)(ZP)

REP 3
REP god -god
REP ster st
REP oo o
```

```
SFX AA Y 5
SFX AA i eg i
SFX AA i ega i
SFX AA i em i
SFX AA i emu i
SFX AA i im i

[ . . . ]
```

Ovdje prvo treba objasniti `CHECKCOMPOUNDPATTERN`. Direktiva je to koja se u ovom slučaju sastoji od dva retka. U prvom je broj pravilā koji slijedi u nastavku, a drugi redak čini samo pravilo: ako jedna sastavnica složenica završava na `-o` i druga počinje na `-o`, tada nemoj prihvaćati `-oo-` nego slaganje provedi samo s jednim `-o-`. Pravilo je uvedeno kako bi se spriječilo sustav da prihvaća nepostojeći oblik `stoogodišnji` umjesto `stogodišnji`.

Direktiva i skup pravila `COMPOUNDRULE` sadrže uzorke na koji način kombinirati sastavnice iz rječnika kako bi se dobili ispravni brojevi. Zastavice se ovdje određuju proizvoljno, ali prvi dio sastavnice mora biti u rječnik upisan tako da bude pogodan za slaganje novih brojeva jer pravila idu kaskadno i počinje se od manjih brojeva prema većima.

`hr_HR.dic`

```
jedn/ZNZW
dv/ZNZW
tr/ZNZW
četver/ZNZW
pet/ZNZW
šest/ZNZW
sedm/ZNZW
osm/ZNZW
devet/ZNZW
deset/ZNZW
jedanaest/ZNZW
dvanaest/ZNZW
trinaest/ZNZW
petnaest/ZNZW
šesnaest/ZNZW
sedamnaest/ZNZW
osamnaest/ZNZW
devetnaest/ZNZW
dvadeset/ZMZNZW
trideset/ZMZNZW
četrdeset/ZMZNZW
pedeset/ZMZNZW
šezdeset/ZMZNZW
sedamdeset/ZMZNZW
```

```
osamdeset/ZMZNZW
devedeset/ZMZNZW
sto/ZLZMZNZW
dvje/ZKZW
tri/ZKZW
četiri/ZKZW
pet/ZKZW
še/ZKZW
sedam/ZKZW
osam/ZKZW
devet/ZKZW
tisuću/ZKZLZMZNZW
o/ZOZW
godišnji/AAZPZW
```

Ovaj je primjer relativno teško za pratiti zbog velikog broja zastavica na istim riječima, pa će se pokušati pojednostaviti što se više može: svi unosi imaju zastavicu ZW, što znači da su pod direktivom ONLYINCOMPOUND kako bismo osigurali da se mogu pojaviti samo unutar složenica jer su unosi u rječničkoj datoteci zapravo krnji. Dakle, ideja je bila sljedeća:

```
ZN    ZO    ZP
jedn  o     godišnji   →     # jednogodišnji

ZM          ZN          ZO          ZP
dvadeset    jedn        o           godišnji   →     # dvadesetdvogodišnji
```

No budući da se na broju dvadeset zadržava i zastavica/klasa naziva ZN, moguć je i oblik

```
ZM          ZO    ZP
dvadeset    o     godišnji   → # dvadesetogodišnji
```

Razlog zašto se na većim brojevima (gdje treba) zadržavaju i zastavice izvorno dane na manjim brojevima, upravo i jest da se mogu pisati brojevi i `dvadesetdvogodišnji`, ali i jednostavno `dvadesetogodišnji`. Ovaj prijedlog jest testiran, ali trebalo bi ga implementirati u rječnik i biti temeljitiji i imati već broj primjera. Nisu se pisale zastavice za brojeve veće od tisuću jer rijetko (ako itko ikad) dolaze riječi kao `dvijetisućestopedesettrogodišnji`... U teoriji jest moguće, ali ovo nisu znamenke da se jednom zvjezdicom naprave svi prirodni brojevi. Uostalom, teorijski prostor postoji, dodaje se još zastavica i unatrag se prati kako zastavice moraju biti posložene u `COMPOUNDRULE`. No što su brojevi veći, to je složenije pisati pravila. Pokušalo se tako graditi numbertext.org[12] za hrvatski, ali čovjek u glavi jednostavno ne može držati sve odnose za velike brojeve. Do tisuću bi trebalo biti dovoljno.

Ovako složena pravila rade za sve brojeve na kojima su se pravila testirala, ali to nije dovoljno. Trebalo bi generirati sve oblike koje `COMPOUND` može stvoriti i ručno provjeriti rade li pravila kako se očekuje. Sumnja je da sada pravila generiraju i nepostojeće oblike ili neki postojeći oblici

[12] https://numbertext.github.io/

nisu pokriveni. Također, pravila će dopustiti i `00-godišnji`, ali na to u ovako postavljenim pravilima ne možemo utjecati. Postoji mogućnost da se umjesto `COMPOUNDBEGIN` i `COMPOUNDEND` koristi isključivo `COMPOUNDRULE`, ali to bi dodatno kompliciralo pravila jer bi znamenki `0` trebalo dodijeliti posebnu zastavicu i unutar direktive COMPOUNDRULE zabraniti da se nula nalazi na početku složenice. Takav način označavanja trebalo ručno provesti sve do broja tisuću.

Ova direktiva ima još jednu specifičnost: provjerava korijene u rječničkoj datoteci odozgo prema dolje i nije moguće imati dva ista korijena za dvjema različitim zastavicama:

```
pet/YAZW
pet/YBZW
```

Ako se brojevi unose ovako u datoteku, Hunspell će uzeti samo prvu riječ, nikada neće doći do druge. Ideja je bila rastaviti brojeve dalje: `pet + naest + o + godišnji` te `pet + o + godišnji`, ali to jednostavno nije moguće zbog tehničkih ograničenja provjernika, koji će uvijek vidjeti samo prvi korijen i pokupiti samo tu zastavicu i nikada neće krenuti dalje. Ovome se može doskočiti ako bi se na isti korijen dodale obje zastavice: `pet/YAYBZW`, ali tada je vrlo teško pohvatati sve korijene i voditi računa koje i koliko zastavica dodijeliti takvome korijenu. Zbog navedenoga se odlučilo da se brojevi u rječničku datoteku upišu kao `petnaest`, odnosno `pedeset` te da im se dodijeli više zastavica ovisno o poziciji na kojoj mogu stajati. Time se izbjeglo dupliranje korijena a sa različitim zastavicama i smanjila se mogućnost pogreške u odnosu na granularnija pravila, ali koja je puno teže pisati i ispravljati.

Teško je uopće odrediti imaju li pravila iz primjera nedostataka jer ne postoji alat koji bi iz ovih pravila generirao sve moguće oblike. Dostupni se alati bave samo prefiksima i sufiksima, slaganje je riječi iz njih potpuno izostavljeno. Za njihovo testiranje potrebno je imati vlastiti alat, ali trenutačno nismo u mogućnosti tako nešto napraviti. Skripta `dictman.php` iz repozitorija https://github.com/krunose/hunspell-hr također ne može generirati složenice jer je rađena u trenutku kada hrvatski rječnik za računalnu provjeru riječi nije imao ugrađene značajke `COMPOUND`, a nema je ni sada. Zbog složenosti zadatka i činjenice da skripta nije unaprijed pripremljena za tu zadaću, teško je se može prilagoditi, osim da se napiše ispočetka. Pravila iz primjera samo su teorijski prikaz onoga što se može (ili bi se moglo), ali bez ozbiljnijega testiranja ova pravila neće biti uključena u živi rječnik jer bi ga mogla učiniti nestabilnim.

Radi jednostavnosti i razumljivosti pravila, dani su primjeri samo sa pridjevom *godišnji*. Međutim treba proći kroz cijeli rječnik i izvući svaku riječ u rječniku uz koju stoji broj. Ovo se može ugrađenim alatima na operacijskom sustavu GNU/Linux, a najprimjereniji alat je SED; i on omogućava pretraživanje rječnika tako što ispisuje retke u kojima se javljaju riječi s unaprijed postavljenim uvjetima. Ako se ovom alatu zada da izdvoji sve riječi koje počinju sa `jedno-`, `dvo-`, `tro-`, `četvero-`, `peto-` ... dobit će se popis na kojemu će se nalaziti -član, `-dijelni`, `-dimenzionalni`, `-glasje`, `-satni`, `-sobni`, `-minutni`, `-mjesečni`, `-godišnji`, `-jezični` i tako dalje. Neke od ovih riječi mogu imati bilo koji broj ispred sebe (poput `-godišnji`), no neke primaju brojeve kao što su jedan ili dva, ali teže primaju veće brojeve (poput `-dimenzionalni` ili `-jezični` ili `-glasje`). Postavlja se pitanje treba li dozvoliti svim takvih riječima da potencijalno

primaju sve brojeve ili treba razdvojiti i ograničiti s obzirom na značenje takvih složenica i činjenice da u korpusima za neke takve spojeve nema (baš) potvrda, recimo `*tridesetdvoglasje`.

Rad s Hunspellom iz naredbenoga retka na GNU/Linuxu

Testiranje rječnika i pravila izrazito je važan dio ovoga posla, no alata za automatsku provjeru kronično nedostaje. Trenutačno nemamo pristup drugim otvorenim bazama riječi ili korpusima koje bismo jednostavno mogli integrirati s uobičajenim alatima na operacijskom sustavu GNU/Linux. Ako kakva baza i jest otvorena, ne može joj se pristupiti bez prijave i nedostaju alati za automatsko preuzimanje podataka kao i alati za automatsku daljnju obradu. Budući da nismo u mogućnosti sami razvijati takva kompleksna rješenja, oslanjamo se na ono što imamo – izvorni Hunspell iz naredbenoga retka na operacijskome sustavu GNU/Linux. Paradoks je u tome što mi nemamo alate za provjeru pri izradi rječnika za računalnu provjeru ispravnosti riječi, ali drugi alati za provjeru i testiranje koriste upravo ovaj rječnik. S obzirom na navedeno, pravila za pisanje brojeva uz imenice i pridjeve iz prethodnoga poglavlja testirala su se tako što se u običnu tekstnu datoteku upisalo stotinjak što točnih što pogrešnih primjera i pokrenula se računalna provjera riječi iz naredbenoga retka narednom

```
hunspell -d hr_HR testnaDatoteka.txt
```

Hunspell je kao alat dostupan i na operacijskom sustavu Windows, no kako je riječ o programu otvorenoga koda koji je primarno razvijan za operacijske sustave slične Unixu, distribucija operacijskoga sustava GNU/Linux činila se kao logičan izbor.

Prethodna naredba pokreće Hunspell u tzv. interaktivnome načinu rada, što znači da će naredbeni redak korisniku predstaviti pogrešno napisane riječi i ponuditi mu moguća rješenja. Prvi dio naredbe poziva program, zastavica `-d` kao unos prima naziv rječnika kojim se kakva tekstna datoteka želi provjeriti; u ovom slučaju je to hr_HR (bez oznaka je li riječ o rječničkoj ili datoteci afiksā) i posljednji argument naredbe je naziv tekstne datoteke koja se provjerava.

Da bi računalni provjernik riječi Hunspell mogao pristupiti rječniku tom naredbom, on se mora nalaziti na standardnoj putanji kojoj provjernik može pristupiti. Na operacijskom sustavu GNU/Linuxu to je najčešće `/usr/share/hunspell`, a to znači da rječnik s https://github.com/krunose/hunspell-hr treba postaviti (samo datoteke `hr_HR.dic` i `hr_HR.aff`) na to mjesto koristeći

```
sudo cp hr_HR.* /usr/share/hunspell/
```

Naravno, prvo datoteke treba preuzeti s mrežnoga spremišta (engl. *repository*), ući u datoteku s preuzetim sadržajem i zatim pokrenuti naredbu.

Često se na spomenutoj putanji nalazi funkcionalan rječnik, a promjene i testiranja se rade na kopiji rječnika na nekom drugom lokalnom mjestu; da se što ne zakomplicira, umjesto postavljanja rječnika na standardnu lokaciju na razini operacijskoga sustava, provjeru je moguće provesti i na kopiji rječnika iz koje druge mape naredbom

```
hunspell -d ./hr_HR testnaDatoteka.txt
```

Tek kada se detaljno provjeri da pravila i ispravci rade kako je zamišljeno, moguće je rječnik kopirati na zadanu putanju operacijskoga sustava, ali i zatražiti uključivanje izmjena u rječnik na GitHubu (*Pull Request*).

Ponekad se u testiranju želi od Hunspella zatražiti da jednostavno izlista sve pogrešno napisane riječi u kakvoj testnoj datoteci, a to se može naredbom

```
hunspell -d ./hr_HR -l < testnaDatoteka.txt
```

Isti taj popis možemo spremiti i u tekstu datoteku kako bi ga provukli još kroz dodatne provjere i analize zašto određeni oblici nisu prihvaćeni. Diferencijskom usporedbom ulazne i izlazne datoteke, recimo, moguće je provjeriti koji oblici su prihvaćeni, a ne bi trebali biti:

```
hunspell -d ./hr_HR -l < testnaDatoteka.txt > rezultatProvjere.txt
```

Ako ovaj rječnik ikada bude označen, moći će se koristiti kao morfološki analizator naredbom

```
hunspell -d ./hr_HR -m
```

Ovaj alat na većini distribucija GNU/Linuxa nije instaliran po zadanim postavkama, a na onima baziranima na operacijskom sustavu Debian može ga se instalirati naredbom

```
sudo apt-get install hunspell && sudo apt-get install hunspell-tools
```

Kada je u pitanju računalna provjera riječi na otvorenim sustavima, Hunspell je apsolutni standard, ali kako aplikacije ne koriste ovaj računalni provjernik na razini operacijskoga sustava, nego ga integriraju po svojim pravilima, nemoguće je predvidjeti rezultate unutar pojedine aplikacije. Budući da se na treće aplikacije nije moguće osloniti, jedini pravi način za testiranje implementiranih novih pravila jest sam Hunspell u naredbenom retku. Uz to, nedostatak adekvatnih popratnih alata i rječničkih baza korisniku ne ostavlja ništa doli vlastita znanja i standardni set GNU alata na otvorenom operacijskom sustavu kao što je GNU/Linux, a što je Hunspellovo prirodno okruženje. Sve to predstavlja veliku prepreku novim suradnicima koji bi željeli pridonijeti razvoju rječnika. Ovaj zadatak zahtijeva ne samo lingvističko znanje već i napredno snalaženje u sistemskom okruženju. Unatoč tome, ovakav je pristup jedini jamac dugoročne stabilnosti rječnika i neovisnosti o nedostatcima pojedinačnih implementacija.

Dodatne napomene uz računalnu provjeru riječi hrvatskoga jezika

Zadaća ovoga dokumenta nije detaljno dokumentirati sam Hunspell, a nije moguće ni pobrojiti sve zamke s kojima se održavatelj i sastavljač ovakvoga rječnika mogu susresti, njegov je zadatak budućim suradnicima dati prvu informaciju i predodžbu onoga što znači raditi na ovakvom projektu. Ove napomene nisu završen ili kompletan popis pravila ili zamki, već kratak uvod koji bi trebao predstaviti općenit smjer i pobrojiti najvažnija saznanja do kojih se došlo nakon godinā održavanja rječnika.

Što uvrstiti u rječnik, a što ne – i zašto

Prvo pitanje koje si održavatelj rječnika za računalnu provjeru pravopisa treba postaviti jest što uopće u rječnik uvrstiti, a što ne. Naime, jednojezični rječnici hrvatskoga jezika teže sveobuhvatnosti. Pravopisni rječnici sadržavaju riječi s kojima je vezan kakav pravopisni problem, poput pisanja glasova *č* i *ć*, *ije* i *je* i tako dalje. Međutim što uvrstiti u rječnik za računalnu provjeru pravopisa? Na prvu bi se moglo zaključiti da treba unijeti sve jer što je više riječi, rječnik je bolji. Pogrešno! Što je više riječi u rječniku za računalnu provjeru riječi, to je veća vjerojatnost da neka pogreška u pisanju izgleda isto kao i riječ koja jest ispravna. Često se umjesto *tako*, piše *tkao*: u ovom slučaju riječ je samo o zamjeni mjesta slova *a* i *k*. Činjenica jest da u rječniku treba biti glagol *tkati*, ali i prilog *tako*, međutim unošenje obiju riječi u rječnik dozvoljava da, ono što je u konkretnom kontekstu zatipak, prolazi neprimijećeno jer su obje riječi upisane u rječnik. Što je više riječi u rječniku, veća je vjerojatnost pojavljivanja ovakvih situacija.

Sadašnji rječnik za hrvatski jezik od glagola radi i aorist; od glagola *čuti*, drugo i treće lice jednine glase *ču*, a prvo lice jednine pomoćnoga glagola *htjeti* je *ću*: budući da korisnici ovakvoga rječnika žele da rječnik ispravlja pogreške u pisanju slova *č* i *ć*, a koja su ujedno i blizu na tipkovnici, dolazi do problema jer rječnik za provjeru riječi jednostavno nije u mogućnosti razlikovati što je u ovom slučaju zatipak zato što je oboje, doslovno, uneseno u rječnik. Tako rječnik propušta *ču* u rečenici *Ja ču sutra učiti*. Ovdje je nužno spomenuti i Levenshteinova udaljenost jer ona, samo ukratko, matematički računa udaljenost jednog znakovnoga niza (riječi) od drugoga. Što je više riječi u rječniku pojedina zamjena mjesta slova (kao u primjeru), brisanje slova (nepritisnuto slovo na tipkovnici) ili umetanje (zabunom) rezultira oblikom koji postoji u rječniku, a provjera pravopisa je ne može uhvatiti i zatipak prolazi neprimijećeno. Ovo je najvidljivije u riječima od, bez detaljne analize, dva do pet slova. Ovakvih problema ima i u većim riječima: *zamijeni* (glagol: *zamijeni ova dva slova*.) ~ *zamjeni* (imenica: *Takvoj zamjeni teza ovdje nema mjesta*.)

Navedeno nameće pitanje treba li u rječnik za računalnu provjeru pravopisa uvrštavati rijetke glagolske oblike koji se u prosječnom tekstu ne korist jer u neku ruku opterećuju rječnik i propuštaju pogreške. Bolje je koju riječ podcrtati i ostaviti na korisniku da procijeni je li riječ o lažnoj uzbuni (engl. *False positive*) ili je zbilja zatipak. Danas su pravopisni rječnici i jednojezični rječnici dostupni na Internetu i takvu je provjeru vrlo jednostavno napraviti. Ipak, označi li se ovaj rječnik i napravi li se kakav lokalni jezični model, ovo neće biti problem jer će provjera gramatike jezičnim modelom znati razliku u ove dvije konkretne situacije.

Ovdje treba postaviti i pitanje treba li u ovakav rječnik unositi samo standardne riječi ili se može propustiti i koja riječ koja pripada razgovornom jeziku, odnosno treba postaviti pitanje koncepta takva rječnika. Budući da se pišu tekstovi različitih namjena u različitim stilovima hrvatskoga standardnoga i nestandardnoga jezika, treba postaviti kriterije po kojima će se odlučivati koja riječ treba, a koja ne treba ući u rječnik. Ovakav rječnik bi trebao služiti za provjeru pri pisanju elektroničke pošte (osobna i poslovna komunikacija), internetskih i novinskih članaka, uređivanje tekstova kakvi se nalaze na Wikipediji, pisanje seminarskih i drugih završnih radova i slično. Ako je opseg rječnika preširok, recimo književno-umjetnički tekstovi, onda se koncept ne može postaviti jer u rječnik mora ući sve, a kada se sve i unese, dolazi do propuštanja rječnika, a time on gubi svrhu. Da, ovo je moguće riješiti izradom jezičnoga modela koji se izvršava lokalno, ali nije

vjerojatno da ćemo to u skorije vrijeme dočekati, zato je nužno postaviti koncept rječnika kroz njegovu namjenu, a onda odrediti što u rječnik treba ući.

U ovakvom rječniku se treba nalaziti riječi koje pripadaju standardnom jeziku, ali i one koje nisu dio strogoga standarda (*glazba ~ muzika*) ako su dovoljno česte u upotrebi jer bez njih rječnik nije od koristi pri pisanju privatnih poruka elektroničke pošte. Jednojezični rječnici ne uvode imena robnih marki, ali u rječniku za računalnu provjeru riječi to je nužnost. Jednojezični rječnik nije opterećen pragmatikom upotrebe jezika, ali rječnik za računalnu provjeru riječi to svakako jest.

Rječnik ne može sadržavati jezik svih struka jer bi ga to opteretilo i korisniku davalo nerazumljive uskostručne prijedloge koji se izvan te struke ne koriste. Svaki računalni rječnik treba imati jezgru koja bi se sastojala od općeg leksika i stručnih termina onoliko koliko je potrebno za pisanje prosječnoga novinskoga ili wikipedijskoga članka. Sve ostalo treba biti dijelom dodatnih suplementarnih rječnika koji se mogu pripremiti kao rječnici pojedinih struka: medicinski rječnik, rječnik pravnoga ili kemijskoga nazivlja i tako dalje.

Iz navedenoga proizlazi da rječnik za računalnu provjeru riječi ne može biti preslika jednojezičnoga rječnika hrvatskoga jezika, ali ni preslika pravopisnoga rječnika jer se u prvom slučaju gubi njegova svrha i ne može se postaviti jasan koncept rječnika, a u drugom slučaju se postiže upravo suprotan efekt od onoga koji bi računalni provjernik trebao imati: ne propuštati zatipke. Računalni provjernik treba biti balans između (stroge) jezične norme i pragmatike svakodnevnoga pisanja u različitim oblicima komunikacije (teksta poruka, elektronička pošta), ali vodeći računa o tehničkim ograničenjima i zakonitostima algoritma za prepoznavanje zatipaka. Uvrštavanje novih riječi zahtijeva opreznost jer ono što je danas česta riječ, sutra već može nestati iz rječnika (kao *koronavirus*), a jednom kada se takva riječ unese u rječnik, teško da će itko poslije iz njega i ukloniti. Zadaća održavatelja nije dati punu rječničku bazu hrvatskoga jezika, već osigurati sigurnost pri pisanju. Ograničavanjem općega leksika i otvaranjem prostora za modularne, uskostručne suplementarne rječnike, računalni provjernik riječi zadržava svoju svrhu – biti pouzdan alat koji pomaže, umjesto da prešućivanjem grešaka korisnika ostavlja u zabludi.

Nekome se može činiti da se balansira između dviju pozicija: jedna podrazumijeva rječnik za provjeru ispravnosti riječi u tradicionalnom smislu, a kada je potrebno ograničiti opseg riječ da se spriječe propuštanja; druga je rječnik kao poligon za treniranje jezičnih modela i kada rječnik treba težiti sveobuhvatnosti. Međutim predlaže se da se sastavi rječnik za provjeru ispravnosti riječi u tradicionalnom smislu, ali da se označi kako bi se njime mogao trenirati manji jezični model otvorenoga koda koji bi se mogao izvršavati lokalno i služio bi kao podloga za uvođenje provjere gramatike alatom LanguageTool[13]. Ako bi se rječnik gradio isključivo za prvi slučaj, nedostatan je za provjeru gramatike, ako bi se radio za drugi, izlazi se iz domene računalne provjere riječi i ulazi u nešto za što ovaj provjernik nije dizajniran.

Tvorba naspram fleksije i granularnost

Već je bilo govora o tome kako se rječnik za računalnu provjeru riječi Hunspellom temelji na dvije datoteke: jedna je rječnička (hr_HR.dic) druga sadržava afikse (hr_HR.aff), koji se, zbog di-

[13] https://languagetool.org/

rektive `FLAG long`, dvoslovnim oznakama dodjeljuju osnovama u rječničkoj datoteci. Ako bi rječnik osim deklinacije i konjugacije sadržavao i tvorbene nastavke, otežalo bi se održavanje jer se gubi granularnost. Ako se prefiksalna tvorba rješava izravnim unosom riječi u rječničku datoteku, zašto bi se sa sufiksalnom radilo drugačije? Ovakav pristup nije sustavan jer tu su još i prefiksalno-sufiksalna tvorba, tvorba slaganjem, ali i složeno-sufiksalna tvorba. Pokriti sve to datotekom afikasa nije uputno. Tu je još i konverzija, ali ona nije relevantni tvorbeni način za računalnu provjeru riječi jer se tiče računalne provjere na razini rečenice. Treba razmisliti kako konverziju riješiti prilikom pripreme rječnika za treniranje lokalnoga jezičnoga modela. Tvorba u datoteci afikasa održavatelju onemogućuje predvidljivost pri unošenju novih riječi jer mora voditi računa i o deklinaciji, ali i derivaciji. Ako postoji klasa u kojoj se istovremeno rješava deklinacija, a dodana joj je i sufiksacija, ona će radi samo za manji broj riječi, ne za sve riječi u cijelom deklinacijskom obrascu jer ne mogu sve riječi tvoriti nove riječi istim derivacijskim obrascem; tako rječnik ima veći broj zapravo istih klasa kojima je jedina razlika u derivaciji, a čime dolazi do ponavljanja, a sve bi se moglo riješiti unošenjem riječi u rječničku datoteku.

Trenutačno u rječnik nije uvedena značajka `COMPOUND`, ali kada se uvede za imenice i pridjeve uz brojeve, prijedlozi ispravaka na riječi `dvadesetogodišnji` neće biti dobri kao kada bi se ta riječ nalazila u rječničkoj datoteci. Pogreške u dva slova pri ovakvom pristupu rezultirat će neočekivanim prijedlozima, ali u suprotnom bi Hunspell i za tako veliku razliku (Levenshteinova udaljenost) na prvom mjestu dao dobar prijedlog. Što je veći dio riječi u rječničkoj datoteci, prijedlozi su bolji. Naravno, u slučaju imenica i pridjeva uz brojeve dodavanje svih brojeva u rječničku datoteku nije dobro jer se u tom slučaju ponavljanje prenosi u rječničku datoteku: *dvadestjednogodišnji*, *dvadesetdvogodišnji*, *dvadesettrogodišnji*... i tako do barem tisuću. U ovom slučaju je zbog specifičnosti potrebno napraviti odstupanje od pravila. Jednom kada se pitanje brojeva uz imenice i pridjeve riješi na predložen način, ova se situacije neće ponavljati jer neće biti naknadnih dodavanja u rječnik.

Granularnost je najvažnija značajka pri održavanju rječnika za računalnu provjeru ispravnosti riječi jer rješavanje komparativa i superlativa datotekom afiksā i primjenom takve klase na sve pridjeve dolazi do propuštanja neispravnih oblika poput *najoptimaliji*. Riječ *optimalno* dolazi od lat. *optimus* što znači *najbolji*, ako se od toga tvori superlativ, dobiva se redundantnost u značenju, što je sa strogo standardnološkoga stajališta pogrešno. Sada u su u rječničku datoteku unošeni pozitivi i komparativi, a od komparativa se tvori superlativ. Sa stajališta granularnosti, ova situacija u sadašnjem rječniku i nije toliko strašna jer se jednostavno može ukloniti *optimalniji* i rječnik više neće moći stvoriti superlativ. Međutim, pogrešno je s aspekta sustavnosti: niti je ovaj problem u potpunosti riješen rječničkom datotekom niti je riješen datotekom afiksā. U slučaju komparacije pridjeva potrebno je napraviti odstupanje od pravila o neprovođenju tvorbe datotekom afiksā da bi se spriječilo posebno unošenje riječi u pozitivu, posebno u komparativu, posebno u superlativu, ali je i jednoj i drugoj i trećoj riječi potrebno dodijeliti odgovarajuće deklinacijske obrasce. Prijedlog je da se stupnjevanje ipak provodi datotekom afiksā, a da se pridjevima poput *optimalan* jednostavno dodaju samo deklinacijske klase, ne i tvorbene.

Ovo se pitanje može širiti i na tvorbu svršenih i nesvršenih glagola: zašto tvorbom iz datoteke afiksā dodavanjem nastavka *-ivati* ne tvoriti nesvršeni glagol od svršenoga *dodijeliti*? Zašto ne datotekom afiksā od *dijeliti* tvoriti *dodijeliti* dodavanjem prefiksa – zato što nije moguće ispratiti

sve kombinacije i zato što različiti glagoli imaju različit set prefiksa i sufiksa i time se nepotrebno usložnjava održavanje rječnika i narušava njegova granularnost. Ako nije riječ o brojevima uz imenicu ili pridjev i nije riječ o stupnjevanju pridjeva, riječi se upisuju u rječničku datoteku, a datotekom afiksā treba rješavati isključivo deklinaciju i konjugaciju. Rječnik ne prati ni tvorbu kako je opisana u gramatikama. Na riječi se dodaju nastavci koji nisu pravilno rastavljeni na tvorbene i gramatičke morfeme, nego se pokušalo nečemu doskočiti, ali nije dokumentirano čemu, a neke su klase potpuno nerazumljive:

```
SFX UF Y      30
SFX UF lac    očev       lac
SFX UF lac    očeva      lac
SFX UF lac    očevu      lac
SFX UF lac    očevim     lac
[ . . . ]
```

Nije mi jasno koja to riječ ili riječi završavaju na `-lac` da im treba dodavati `-očev`, `-očeva`, `-očevu`, `očevim`. Ako je suditi prema onome što se može vidjeti u rječniku, ova je klasa dodijeljena riječima *gringo*, *potrčko* i *žiro-račun*. Problema je više jer *gringo* uopće ne treba unositi u rječnik. Riječ *žiro-račun* je pogrešno unesena jer pravopis Instituta za hrvatski jezik i jezikoslovlje bilježi da se piše *žiroračun*, a ova klasa na riječi *potrčko* je sasvim nepotrebna jer *potrčko* ne završava na `-lac` pa se klasa i ne aktivira. Ovaj deklinacijski obrazac je dodijeljen i, u sklopu druge kompresije alijasima, riječima *nestaško*, *šašavko*, *tupavko* i još nekolicini. Ni u jednom se slučaju klasa neće aktivirati. Klasa XN u rječniku ima 61 unos, što je previše. Takvu je klasu potrebno rastaviti na više manjih dijelova kako bi rječnik postao granularan. Sada je teško procijeniti kako će se nove riječi kojima se ova klasa dodijeli ponašati u praksi. Trenutačno se oslanja isključivo na sastavljačevu sposobnost postavljanja uvjeta. Ovakve su klase nepotrebne i opasne.

Označavanje rječnika za provjeru gramatike

Istina, Hunspell je danas već pomalo relikt nekoga drugoga vremena, ali takvim ga je učinila činjenica da postoji umjetna inteligencija i veliki jezični modeli koji istovremeno provjeravaju i pravopis i gramatiku s dubokim razumijevanjem rečenice, što nijedan klasični alat ne može. Ipak, postoje alati i aplikacije u koje ovakve moderne tehnologije nisu implementirane i tako će ostati do daljnjega: LibreOffice odbija uključiti AI u svoj izvorni kôd zbog zaštite privatnosti korisnika te sprječavanja ovisnosti o velikim tehnološkim divovima. Chrome, iako proizvod Googlea, još uvijek za računalnu provjeru riječi u stvarnom vremenu koristi Hunspell i pitanje je hoće li se to promijeniti jer su i najmanji (možda i lokalni) modeli umjetne inteligencije iznimno teški i troše mnogo resursa, a time traže i prilično snažno sklopovlje (engl. *hardware*). Ipak, LanguageTool je dostupan kao dodatak Chromeu tako da Hunspell može poslužiti kao baza za izradu pravila za provjeru gramatike tim jezičnim alatom, a onda se kao dodatak može uključiti u ovaj internetski preglednik. Tako je Hunspell još uvijek i više nego aktualan i održavanje rječnika za ovaj računalni provjernik riječi izrazito je nužno jer ne samo da još nije prošlost nego su rječnici za računalnu provjeru riječi Hunspellom preduvjet za budućnost.

Međutim, da bi ovakvi rječnici postali alatom za treniranje jezičnih modela, potrebno je imati potpuno označen rječnik (engl. *Part-of-Speech tagging*). Trenutačno u rječniku imamo označene samo infinitive glagola kako bi računalna provjera na razini rečenice LibreOfficeovim ugrađenim alatom LightProofom[14] mogla provjeravati krnje infinitive uz futur prvi, ali ni ti infinitivi nisu dovoljno dobro označeni jer se samo infinitivima dodala oznaka da je riječ o glagolu, nije se provelo puno i potpuno označavanje: označiti infinitiv znači riječi u rječničkoj datoteci dodijeliti oznaku `glagol`, ali onda i oznaku `infinitiv` kako bi provjera gramatike danas-sutra mogla razlikovati glagol u prezentu od infinitiva; to se sada nije napravilo tako jer se nije imalo vremena, ali ni znanja da bi se ovaj proces uspješno priveo kraju. Do sada nije bilo potrebe za označavanjem rječnika, ali danas je to za hrvatski jezik preduvjet pri hvatanju koraka s većim jezicima.

Zašto je potrebno imati označen rječnik

Do sada označen rječnik nije bio nužnost jer umjetne inteligencije nije postojalo, no danas svakako jest. Ovakvi rječnici više ne moraju biti samo provjernici zatipaka, nego poligon za testiranje novih tehnologija. Hunspell jest morfološki analizator, ali ta je značajka sada potpuno neiskorištena. Ne treba se ni zadovoljiti provjerom ispravnosti napisanih riječi u izolaciji kada moderni jezični modeli mogu vršiti provjeru gramatike vodeći računa o kontekstu. Otvorena provjera gramatike za hrvatski ne postoji. LightProof je samo set čistih pravilnih izraza (engl. *Regular expressions*) i ne razumije kontekst. Označen rječnik može pomoći, ali samo to neke mjere jer ovom alatu nedostaju naprednije značajke. LanguageTool te značajke ima, kao i ugrađen jezični model, ali nije treniran za provjeru gramatike hrvatskoga jezika[15] jer nema baze na kojoj bi se to izvelo. Riječ je o industrijskom standardu otvorenoga koda, ali standardu koji ne može naučiti ništa iz sirova teksta, već mu trebaju upute: matematički uzorci velike količine podataka. Treba mu označen rječnik i tekstovi. Rad na označavanju ovoga rječnika više nije puko ispravljanje zatipaka u LibreOfficeu, Firefoxu, Thunderbirdu, Chromeu ili kojem drugom od mnogih aplikacija koje koriste ovu tehnologiju, nego je riječ o činjenici da postoji (otvorena) tehnologija koja stvarno može (i treba) razumjeti hrvatski tekst.

Bez oznaka osuđeni smo na zatvorene modele velikih tehnoloških divova kojima je hrvatski jezik zapravo komercijalno neisplativ, pa onda nerijetko pada u drugi (ili treći) plan. Nije ovo pitanje altruizma, već brige o privatnosti i zaštiti podataka. Nije uputno svaki tekst slati na tuđe poslužitelje na analizu jer neki od njih sadrže povjerljive podatke, a u nekim slučajevima postoji i zakonska obaveza da se to ne čini; recimo GDPR ne zabranjuje tehnologiju u oblaku, ali postoje zabrane da se tekstovi koji sadrže povjerljive podatke šalju u oblak bez izričite privole onoga kome ti podatci pripadaju. Veliki jezični modeli mahom su smješteni u Sjedinjenim Američkim Državama, a Europska unija zabranjuje slati takve podatke na poslužitelje izvan EU-a jer tamo ne postoji isti standard zaštite privatnosti. Čak ako je model i besplatan, koristi li podatke za daljnje treniranje modela, teško krši GDPR. Slanje medicinske dokumentacije na udaljene poslužitelje predstavlja izravno kršenje zakonskih odredbi. Iako se medicinski nalazi nužno ne lektoriraju, isto se odnosi na pravosuđe, odvjetništvo, javnu upravu i policiju, gdje je riječ o tekstovima kojima

[14] https://github.com/krunose/lightproof-hr
[15] https://forum.languagetool.org/t/adding-croatian-language/12096

bi uvelike koristila takva usluga; ne zbog pravopisnih i gramatičkih grešaka, već činjenice da ne smiju napustiti računalo onoga tko piše. Ne bude li hrvatski držao korak sa standardima u novom digitalnom dobu, dugoročno će biti ugrožena njegova funkcionalna stabilnost na službenoj i profesionalnoj razini. Razvoj otvorenih morfoloških baza nije luksuz, nego tehnološki preduvjet da jezik ostane ravnopravan i primjenjiv. Ovaj se problem treba riješiti morfološkim i semantičkim označavanjem Hunspellova rječnika.

Iako internetsko pretraživanje pokazuje da se radi na hrvatskim jezičnim modelima, postavlja se pitanje njihove praktične upotrebljivosti u svakodnevici jer će biti ogromni i zahtijevat će vrlo snažna računala, a takva javna uprava, odvjetništvo, sudstvo i policija ne posjeduju. Iako će takvi modeli biti dobri u smislu znanosti i struke, ovdje postavljam pitanje njihove praktične primjene.

Označavanje rječnika – tehnička strana priče

O tehničkoj strani označavanja Hunspellova rječnika već je kratko bilo spomena u dijelu `Rječnička datoteka – relativno jednostavan popis riječi` te je vrijeme da se ovome posveti koji redak više. Označavanje se provodi i u rječničkoj i datoteci afiksā. Budući da se u prvoj nalazi kanonski popis riječi, u ovoj potonjoj se nalaze (samo) konjugacije i deklinacije, ali označavanje provedeno samo u prvoj znači da će se oznake u procesu morfološke analize izgubiti ako nisu dodane i u datoteku afiksā. Oznake se sastoje od dvoslovnih kratica i dvotočke, dakle iza svake riječi je potrebno troslovnim oznakama i tabulatorom odijeliti jednu oznaku od druge. Uzet će se za primjer glagol *pisati*:

hr_HR.dic

```
1
pisati/AA	po:glagol	is:infinitiv	is:nesvršeni	is:prijelazni
```

Važno je primijetiti nekoliko stvari, prvo oznake u nisu proizvoljne. Oznaka `st:` stoji za (engl.) `stemm`, odnosno predstavlja kanonski oblik riječi. Tako `po:` stoji za (engl.) *part-of-speech*, odnosno vrstu riječi, a `is:` predstavlja (engl.) *inflectional suffix* iliti gramatički nastavak. No upiše li korisnik *pišem*, Hunspell će obrisati sve ove oznake jer *pisati* nije isto što i *pišem*; zbog toga u datoteku afiksā treba unijeti sljedeće oznake:

hr_HR.aff

```
[ . . . ]
SFX AA Y sati šem sati po:glagol is:prezent	is:1.l.	is:jednina	is:nesvršeni	is:prijelazni
[ . . . ]
SFX AA Y sati ao sati	po:glagol	is:glagolski_pridjev	is:muški_rod	is:jednina
[ . . . ]
```

Na ovaj će način provjernik znati da je *pišem* glagol u prezentu u prvom licu jednine, da je riječ o nesvršenom prijelaznom glagolu. Oznake je, kako je već rečeno, potrebno ponoviti jer će za oblik *pisati* Hunspell pokupiti dodijeljen set oznaka uz taj oblik, ali ne može oznake spajati, nego

će za *pišem* obrisati sve što je bilo uz *pisati*, pa je nužno dodati oznake i u datoteku afiksā kako bi prezent dobio svoje oznake. S tehničke strane ova priča nije kompliciranа, ali nismo u stanju ovaj zadatak riješiti sami jer nema baze prema kojoj bi se provela automatska (programska) dodjela morfoloških oznaka. Posao je to koji nadilazi tehničke, intelektualne i vremenske mogućnosti pojedinca.

Ponavljanje oznaka u datoteci afiksā nije nedostatak, nego se na to može gledati kao na značajku. Ako se označe klase u datoteci afiksā, recimo za glagole prve vrste, rječnik će automatski svim glagolima kojima je ta klasa dodijeljena dodati i oznake koje uz nju idu. Na ovaj se način jednim potezom rješava i tisuće riječi u rječniku. Možda bi bolje bilo prvo označiti, jednom kada se pročisti, datoteku afiksā jer se tako jednim potezom označava većina rječnika, a potom je potrebno proći kroz rječničku datoteku i ponoviti postupak za kanonske oblike.

U ovom je trenutku potrebno postaviti i pitanje homonima u rječniku: trenutačno ovaj dio nije dobro riješen, zapravo nije riješen uopće jer je rječnik lišen, nažalost, svih duplih unosa. Da bi morfološko označavanje rječnika imalo smisla, potrebno je u rječniku imati sljedeće:

`hr_HR.dic`

```
3
kosa/AA po:imenica [...]
kosa/AB po:imenica [...]
kosa/BA po:pridjev [...]
```

U prvom slučaju je riječ o imenici u značenju *vlasi*, druga pak u kontekstu košnje trave, a posljednja je pridjev. Ovo je jedini smisleni način označavanja rječnika. Tvorba konverzijom ovdje bi mogla raditi ozbiljne probleme, recimo: riječi na -ski kao pridjev i kao prilog. Bilo je nužno spomenuti ovaj problem, ali dublje se u njega neće zalaziti jer nadilazi opseg i zadaću ovoga teksta.

Budući da jedna osoba sama ne može označiti rječnik, treba voditi i računa o suradničkoj strani vođenja izmjena. Alat `Git` bi ovdje bio od velike pomoći, ali izmjene na istim dijelovima teksta od strane više suradnika rezultirat će sukobom u kodu (engl. *merge conflict*); najbolje bi bilo razdijeliti rječnik na manje dijelove, raditi na njima, a onda ih spojiti nakon označavanja. Možda postoje i prigodniji alati za ovakav posao, ali ne znamo za njih i radimo s alatima otvorenoga koda na otvorenom operacijskom sustavu GNU/Linux.

Označavanje rječnika – nama neistražen teritorij

Označavanje rječnika je front koji se neće otvarati. Godinama se dodaju nove riječi i pokušava se čistiti baza, ali taj posao uzima toliko slobodnoga vremena da je označavanje rječnika nedostižan zadatak. Primjeri o označavanju rječnika u ovom dokumentu nisu putokaz kako označavati i koje oznake koristiti, već im je zadaća pokazati tehničku stranu označavanja. Vrste oznaka ovise o alatu za provjeru gramatike za koji se rječnik prvenstveno priprema, a treba voditi i računa o standardnima označavanja za slavenske jezike. Budući da postoji MULTEXT-East (Multext) morfosintaktička specifikacija (MSD), moguće je rječnik označiti na taj način unutar oznake is: u

rječniku. Naše su znanje i mogućnosti ovime što je ovdje navedeno iscrpljene i ovaj se zadatak ostavlja onome tko ovom materijom barata bolje.

Samo ukratko za kraj

Ovaj dokument nije nastao iz želje da se napiše tehnički priručnik. Nastao je s ciljem da se osigura kontinuitet održavanja rječnika za računalnu provjeru riječi Hunspellom. Organski rast projekta uzrokovao je određene nedosljednosti u njegovu kodu i te je nedosljednosti potrebno ispraviti; prilagoditi rječnik za potrebe i izazove nove digitalne ere. Projekt ima potencijala postati nešto više, ali to može samo uz temeljito restrukturiranje jer u suprotnom ne može biti više doli relikt prošlosti. Raskrižje na kojemu se ovaj rječnik nalazi zahtijeva predanost i resurse koji – kako u znanju tako i u tehničkim mogućnosti – nadilaze snagu i volju pojedinca. Više nije riječ o želji, nego činjenici da u razvoju sličnih projekata svjetskih jezika izravno sudjeluju institucije koje se bave računalnom lingvistikom. Dodavanje novih riječi ne vodi k napretku, potrebno je iz temelja promijeniti njegovu svrhu. Sadašnje nedosljednosti nisu krivnja sastavljača i održavatelja niti prošlih niti sadašnjih jer nitko početkom dvijetisućitih nije mogao predvidjeti trenutak pojave umjetne inteligencije pa onda ni pripremiti rječnik za nove izazove. Ne uhvatimo li korak s vremenom, hrvatski jezik ostat će bez jedinoga i slobodnoga alata za računalnu provjeru riječi.

Ovaj rječnik nema alternative jer je jedini koji korisniku osigurava četiri temeljne slobode: preduvjet svim slobodama jest mogućnost da se ovaj kod izvršava bez ikakvih ograničenja i u bilo koje svrhe. Sloboda da se izvorni kod rječnika proučava osigurava digitalnu opstojnost maloga jezika kao što je hrvatski. Dijeljenje kopija rječnika osigurava da se ista materija pojavljuje u nebrojnim aplikacijama i drugim računalnim rješenjima kako slobodnima tako i komercijalnima, a posljednja sloboda – ona da se izvorni kod rječnika mijenja – jest ono što je osiguralo kontinuitet razvoja od početka tisućljeća do danas. Infrastruktura postoji, samo je treba razvijati. Već je spomenuto kako ovaj rječnik nema alternative i to je potrebno ponoviti: ne postoji zamjena koja bi osigurala postojanje rječnika za računalnu provjeru riječi koji može raditi na operacijskim sustavima Windows, GNU/Linux, macOS i ChromeOS i popis vjerojatno nije konačan. Broj aplikacija i računalnih rješenja koje već koriste ovaj sustav je gotovo beskonačan; a sada je tu još i mogućnost korištenja ovoga rječnika za razvoj slobodnih jezičnih modela pogonjenih umjetnom inteligencijom. Hrvatski jezik u novom digitalnom dobu ne čuva se iza zatvorenih vrata i prosječnom korisniku nedostupnih jezičnih modela.

Kako bi se osigurala praktična vrijednost rječnika, predstojeći rad mora se odmaknuti od teorije i koncepcije tiskanih jezičnih priručnika jer je jedino mjerilo uporabne vrijednosti ovoga rječnika – praksa. Prvi korak je optimizacija klasa unutar datoteke afiksā kako bi se postavili temelji za označavanje rječnika jer označiti ovu datoteku znači na jednostavan način riješiti najveći dio posla. U bazu se mora uvesti razdvajanje homonima, treba je dopuniti novim pojmovima, ali i nazivima robnih marki. Treba balansirati između krute norme i uporabne vrijednosti rječnika. Pred nama je jasan izbor: ili ćemo prepoznati važnost slobodne jezične infrastrukture, ili ćemo pustiti da jedino što čuva digitalnu pismenost na hrvatskom jeziku postane običan relikt prošlosti.

Za kraj treba zahvaliti Denisu jer bez njega ovaj rječnik ne bi ni postojao. Hvala i Mirku, koji je razradio datoteku afiksā kakvu imamo danas. Hvala i ostalima koji me obavještavaju da rječniku nedostaje ova ili ona riječ.